OFICINA POSTGRESQL — PIE IV

Criando um Banco de Dados do Zero

APOSTILA DO ALUNO — Material Autocontido

UFR — Projetos Integrados de Extensão | ETEC — Desenvolvimento de Sistemas

Bem-vindo à Oficina

Esta apostila é seu guia completo para criar um banco de dados PostgreSQL do absoluto zero. Você não precisa de experiência anterior — cada passo será explicado com detalhes e contexto.

Ao final, você terá construído um banco real, com usuários, permissões e dados populados — pronto para ser consultado na próxima aula com SQL.

Pré-requisitos

- **PostgreSQL 17** instalado no Windows
- **psql** configurado no PATH do sistema
- **DBeaver** instalado (para a próxima aula)

Configurando o PATH do psql

Após instalar o PostgreSQL, o `psql` pode não funcionar no CMD. Para corrigir:

1. Tecla Windows → buscar **Variáveis de ambiente** → abrir
2. Em **Variáveis do sistema**, localizar `Path` → clicar em **Editar**
3. Clicar em **Novo** e adicionar: `C:\Program Files\PostgreSQL\17\bin`
4. Confirmar com OK em todas as janelas
5. **Fechar e reabrir o CMD**

> ⚠ Sempre feche e reabra o CMD após alterar variáveis de ambiente. A janela aberta antes não reconhece as mudanças.

Verificando o serviço PostgreSQL

O PostgreSQL funciona como um serviço do Windows. Abra o CMD e verifique:

```
sc query postgresql-x64-17
```

Se o estado for **RUNNING**, está pronto. Se for **STOPPED**:

```
net start postgresql-x64-17
```

> ⚠ Para iniciar/parar o serviço, abra o CMD como administrador (botão direito → Executar como administrador).

Conceitos Fundamentais

SERVIDOR (INSTANCE / CLUSTER)

É o processo do PostgreSQL rodando na máquina. Pense nele como um **prédio**. Um único servidor pode conter vários bancos de dados.

ROLE

No PostgreSQL, "usuário" e "grupo" são o mesmo conceito — chamados de **role**. Uma role pode ter permissão para login (como usuário) ou apenas agrupar permissões. Pense como um **crachá de acesso**.

BANCO DE DADOS (DATABASE)

Um container isolado dentro do servidor. Cada banco tem seus próprios schemas, tabelas e dados. É a **sala dentro do prédio**.

SCHEMA

Uma pasta organizacional dentro do banco. O schema padrão se chama `public` — é onde criaremos nossas tabelas.

PRIVILÉGIOS (GRANT / REVOKE)

Definem o que cada role pode fazer: ler, escrever, criar tabelas, administrar. São as **permissões do crachá**.

> 💡 Resumo: servidor = prédio | banco = sala | tabela = armário | role = crachá com permissões

---

Conectando ao PostgreSQL via CMD

```
psql -U postgres
```

O prompt mudará para `postgres=#`. Explore o ambiente:

```
\l        -- lista todos os bancos de dados
\du       -- lista todas as roles
\conninfo -- mostra sua conexão atual
```

Por padrão você verá apenas os bancos de sistema (`postgres`, `template0`, `template1`) e a role `postgres`.

Criando as Roles

Trabalharemos com 3 roles com diferentes níveis de acesso:

SUPER ADMIN — CONTROLE TOTAL

```
CREATE ROLE admin_escola WITH
    LOGIN
    SUPERUSER
    PASSWORD 'Admin@2026';
```

LOGIN permite autenticação direta. **SUPERUSER** dá poder irrestrito — equivale a um administrador total.

EDITOR — LEITURA E ESCRITA

```
CREATE ROLE editor_escola WITH
    LOGIN
    PASSWORD 'Editor@2026';
```

Sem `SUPERUSER`. Os privilégios serão concedidos depois com `GRANT`.

LEITOR — SOMENTE LEITURA

```
CREATE ROLE leitor_escola WITH
    LOGIN
    PASSWORD 'Leitor@2026';
```

```
\du   -- verifique as 4 roles criadas
```

> ✔ Você deve ver: postgres, admin_escola, editor_escola, leitor_escola

> ⚠ Senhas em texto plano são apenas para fins didáticos. Em produção use senhas fortes e gestores de segredos.

Criando o Banco de Dados

```
CREATE DATABASE banco_escolar
```

```
    OWNER admin_escola
    ENCODING 'UTF8';
```

`OWNER` define quem tem controle total. `ENCODING 'UTF8'` garante suporte a acentos e caracteres especiais.

Saia e reconecte como admin no novo banco:

```
\q
```

```
psql -U admin_escola -d banco_escolar
```

```
\conninfo   -- confirme: banco = banco_escolar, user = admin_escola
```

> ✔ Conexão confirmada no banco correto com o usuário correto

---

Criando as Tabelas

Vamos montar o **Banco Escolar** — 9 tabelas de um sistema acadêmico.

IMPORTANTE: ORDEM DE CRIAÇÃO

Uma tabela só pode referenciar outra que **já existe**. Por isso criamos as dimensões primeiro, depois as tabelas que as referenciam.

DIMENSÕES (CRIAR PRIMEIRO)

```
CREATE TABLE especialidades (
    id   SERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL
);

CREATE TABLE professores (
    id               SERIAL PRIMARY KEY,
    nome             VARCHAR(100) NOT NULL,
    email            VARCHAR(100) NOT NULL,
    especialidade_id INT NOT NULL REFERENCES especialidades(id)
);

CREATE TABLE turmas (
    id           SERIAL PRIMARY KEY,
    nome         VARCHAR(50) NOT NULL,
    ano          INT NOT NULL,
    semestre     INT NOT NULL,
    professor_id INT NOT NULL REFERENCES professores(id)
);

CREATE TABLE materias (
    id            SERIAL PRIMARY KEY,
    nome          VARCHAR(100) NOT NULL,
    carga_horaria INT NOT NULL DEFAULT 64
);

CREATE TABLE tipos_notas (
    id   SERIAL PRIMARY KEY,
    nome VARCHAR(50) NOT NULL
);
```

GLOSSÁRIO DOS TIPOS USADOS

- `SERIAL` — inteiro que incrementa automaticamente (1, 2, 3...)
- `PRIMARY KEY` — identificador único da linha

- `VARCHAR(n)` — texto com até n caracteres
- `INT` — número inteiro
- `REFERENCES tabela(col)` — chave estrangeira
- `NOT NULL` — campo obrigatório
- `DEFAULT` — valor padrão quando não informado

FATOS E RELACIONAMENTOS (CRIAR DEPOIS)

```
CREATE TABLE alunos (
    id              SERIAL PRIMARY KEY,
    nome            VARCHAR(100) NOT NULL,
    email           VARCHAR(100) NOT NULL,
    data_nascimento DATE,
    turma_id        INT NOT NULL REFERENCES turmas(id)
);

CREATE TABLE matriculas (
    id             SERIAL PRIMARY KEY,
    aluno_id       INT NOT NULL REFERENCES alunos(id),
    turma_id       INT NOT NULL REFERENCES turmas(id),
    data_matricula DATE NOT NULL
);

CREATE TABLE notas (
    id         SERIAL PRIMARY KEY,
    aluno_id   INT NOT NULL REFERENCES alunos(id),
    materia_id INT NOT NULL REFERENCES materias(id),
    nota       FLOAT NOT NULL,
    tipo_id    INT NOT NULL REFERENCES tipos_notas(id)
);

CREATE TABLE presencas (
    id          SERIAL PRIMARY KEY,
    aluno_id    INT NOT NULL REFERENCES alunos(id),
    materia_id  INT NOT NULL REFERENCES materias(id),
    data_aula   DATE NOT NULL,
    presente    BOOLEAN NOT NULL,
    justificada BOOLEAN
);
```

```
\dt   -- confira as 9 tabelas criadas
```

> ✔ 9 tabelas listadas com sucesso

Aplicando Permissões (GRANT)

As roles existem, mas ainda não têm acesso ao banco. O `GRANT` concede permissões específicas.

EDITOR — LEITURA E ESCRITA COMPLETAS

```
GRANT CONNECT ON DATABASE banco_escolar TO editor_escola;
GRANT USAGE ON SCHEMA public TO editor_escola;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO editor_escola;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO editor_escola;
```

O QUE CADA LINHA FAZ

- `CONNECT` — permite se conectar ao banco
- `USAGE ON SCHEMA` — permite enxergar o schema public

- `SELECT, INSERT, UPDATE, DELETE` — operações nas tabelas
- `USAGE, SELECT ON SEQUENCES` — necessário para usar o SERIAL

LEITOR — SOMENTE LEITURA

```
GRANT CONNECT ON DATABASE banco_escolar TO leitor_escola;
GRANT USAGE ON SCHEMA public TO leitor_escola;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO leitor_escola;
```

TESTANDO NA PRÁTICA

```
\q
psql -U leitor_escola -d banco_escolar
```

```
INSERT INTO especialidades (nome) VALUES ('Teste');
```

> ✔ Esperado: ERROR: permission denied for table especialidades

Esse erro é proposital — confirma que a role de leitura não pode escrever. Agora tente como `editor_escola` — o INSERT deve funcionar.

Populando as Tabelas

```
\q
psql -U admin_escola -d banco_escolar
```

Execute o script completo em `dados/04_inserts.sql` do repositório.

Tabela	Registros
especialidades	5
professores	8
turmas	4
materias	6
tipos_notas	3
alunos	20
matriculas	60
notas	240
presencas	960

```
SELECT COUNT(*) FROM alunos;      -- esperado: 20
SELECT COUNT(*) FROM presencas;   -- esperado: 960
```

> ✔ Se os contadores batem, o banco está completo e pronto para a próxima aula.

O que construímos hoje

- ✔ Serviço PostgreSQL verificado e rodando
- ✔ 3 roles criadas com diferentes níveis de acesso
- ✔ Banco de dados banco_escolar criado

- ✔ 9 tabelas com chaves primárias e estrangeiras
- ✔ Permissões testadas e comprovadas na prática
- ✔ 960 registros populados

Próxima aula — Oficina de SQL (PIE V)

Você vai conectar ao `banco_escolar` pelo **DBeaver** e aprender SQL com:

- `SELECT`, `WHERE`, `ORDER BY`, `LIMIT`
- `GROUP BY`, `HAVING` e funções de agregação
- `INNER JOIN`, `LEFT JOIN`, `FULL OUTER JOIN`
- CTEs com `WITH`

Repositório: **github.com/ocarlossoueu/oficinasql**

Cheat Sheet — Comandos psql

Comando	Função
`\l`	Listar bancos
`\du`	Listar roles
`\dt`	Listar tabelas
`\conninfo`	Conexão atual
`\c nome_banco`	Trocar de banco
`\q`	Sair do psql
`\?`	Ajuda dos comandos
`\h COMANDO`	Ajuda de sintaxe SQL